

Week 2 Study Guide: Agent Foundations & Human Review

Lecture 1: Agent Foundations (Slides 1.1–1.17) | Lecture 2: Git & GitHub Workflow (Slides 2.1–2.9) | Bold = key term · *Italic note* = instructor note

LECTURE 1 — Agent Foundations & Human Review (17 slides)

- 1.1 Setup Checkpoint** — Run `claude --version`; sort into categories A–D (ready / error / no access / not started).
- 1.2 Course Site Navigation** — Click three course links; find a real 404 on the Announcements page. Distinguish observable symptoms from unknown causes.
■ *The 404 is genuine — see it live on the projector before anything is described.*
- 1.3 Unverified Response** — Asking AI without the URL → unverified guess. *"Nothing was checked. It never opened the page. Status: unverified."*
■ *Do not require a device; the unverified answers students return are all the point.*
- 1.4 Agent Capability** — **Definition:** "A model that can look, whose next move depends on what it saw, and that can decide to look again, is an agent." Four moves: read → ask permission → look → propose then stop.
■ *Hold step 1 until most pairs have three verbs down.*
- 1.5 Text Message Exercise** — Questions answerable from memory vs. questions requiring real-world observation — tool access enables the instinct to "look it up."
- 1.6 Write Instructions Once** — In pairs, write a 3–5 step broken-link procedure. **Skill** = reusable procedure; **Tool** = one action returning one result.
■ *Hold step 1 for the full three minutes even if the room goes quiet.*
- 1.7 One Look Only** — Three sequential observations each depend on the previous — cannot be planned in advance; a loop is required.
- 1.8 Write a Loop** — Formulate the rule: what to examine, what to do with each answer, when to stop (termination condition).
■ *Real loop demo — ccunpacked.dev, "The Agent Loop" at 0.5x; step 7 = decide, step 8 = repeat, step 11 = stop.*
- 1.9 Role-Play Simulation** — Four roles: Goal Keeper, Model, Host/Tool, Reviewer/Recorder — reveals where human authority boundaries must sit.
- 1.10 Ambiguous Instruction** — "Fix the Announcements link" permits three violations: editing without permission, guessing without evidence, no stop rule. Introduces the **Agent Brief**: Outcome · Evidence · Limits · Stop Rule.
- 1.11 Study Guide Lab** — Write a four-part brief; run against an approved source; verify two claims; request one change. *15-min lab.*
■ *Use the linked worksheet — four-part builder, live/supplied-context/simulated routes, print-preview check.*
- 1.12 Accept the Report?** — Agent claims: *"Fixed the Announcements link and verified it works."* Most accept — the acceptance is the lesson.
■ *Take a show of hands before revealing; record the split.*
- 1.13 Trace Reveals Problems** — Six entries in record. **Entry 5:** edited without permission. **Entry 6:** claimed verification without evidence. *"The report hid both; the record cannot."*
■ *Let them hunt two full minutes before revealing.*
- 1.14 Write Permissions** — Draft a permission making Entry 5 legal: (a) narrow permission + verification, or (b) keep read-only with human application.
■ *Test each permission aloud — what may it change, what must it show, where does it stop?*
- 1.15 Three More Failures** — Study-guide agent: read unauthorized notes · invented unsupported deadline · emailed draft without permission. Each gap needs its own boundary.
■ *Run all three as a quick show of hands before revealing.*
- 1.16 Four Safeguard Layers** — Build boundaries across: (1) Context/Privacy, (2) Tools/Permissions, (3) Evidence/Checks, (4) Stop/Hand-off.
■ *Require enforceable controls — "be safe" is not a control.*
- 1.17 Exit Ticket** — (1) LLM fluency ≠ verification. (2) Agents use skills + tools by looping. (3) Systems must stop and hand off at consequential action points.
■ *Collect two contrasting boundary examples aloud; preview Meeting 2.*

LECTURE 2 — Git & GitHub Workflow (9 slides)

- 2.1 Where Is Your Study Guide?** — Three locations: chat, saved file, agent memory. Reveal: **"Save a file you can reopen."**
- 2.2 Save, Commit, or Push?** — Four locations (file, folder, history, GitHub) → three moves. Only **push** leaves the computer; save and commit stay local.
■ *Point at each of the four places as you read them, then again during demo.*
- 2.3 Git vs. GitHub** — **Git** = local software for checkpoints/version recovery. **GitHub** = website for reaching other computers. Neither requires the other.
■ *Three minutes, immediately before the setup window.*
- 2.4 Workshop Doorway** — External workshop for hands-on Git setup and a shared save/commit/push demonstration.
- 2.5 Approve the Destination** — Before pushing: verify the exact destination AND contents in the browser — a "success message" is a claim; opening GitHub is evidence.
■ *Private repository — pushing does not publish a website; read actual errors aloud.*
- 2.6 Improve One Sentence** — Fix one sentence → create a **diff** (side-by-side before/after). Second checkpoint records the change; update `next-steps.txt`.
■ *Insist on ONE sentence — students who rewrite half can't read their own diff.*
- 2.7 Which Version Online?** — After committing, GitHub still shows the first version. Committing changes local; only pushing changes GitHub. Cycle runs twice to reinforce.
■ *The gap between committing and pushing is the most common beginner surprise.*
- 2.8 Fresh Session Pickup** — New agent session reads `next-steps.txt` to continue. **"Memory is not a file."** Saved text persists; chat memory does not.
■ *This screen justifies the whole meeting — watch a new session read their own note.*
- 2.9 Explain the Workflow** — **Save** = updates file. **Commit** = records locally. **Push** = sends to GitHub. Document where work lives, what changed, next task.
■ *Ask three students to point at where their work lives and say the three words in order.*

Key terms: Agent = model that can look + loop + stop · Skill = reusable procedure · Tool = one action, one result · Agent Brief = Outcome · Evidence · Limits · Stop Rule · Trace = record that hides nothing · Save → Commit → Push